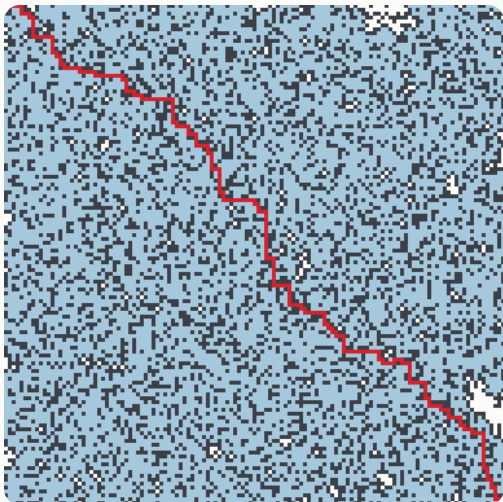
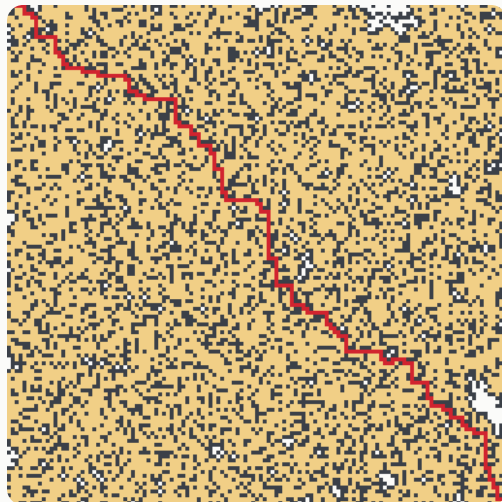


# 격자 지도 경로 탐색 비교

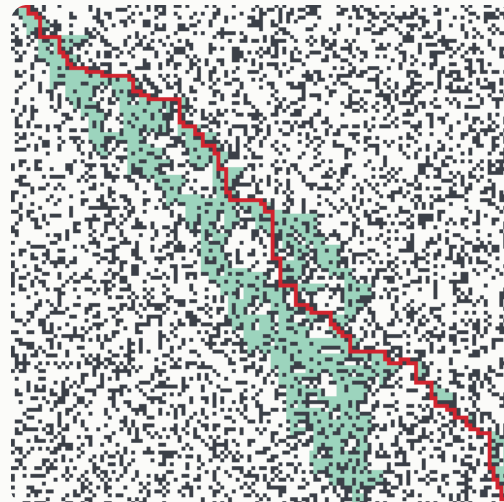
BFS · Dijkstra · A\*



**BFS** 확장 11,409칸



**Dijkstra** 확장 11,409칸



**A\*** 확장 1,833칸

같은 129 × 129 무작위 벽 지도에서 세 방식이 뒀진 칸(색)과 찾은 경로(빨강). 경로 비용은 셋 다 258.

2026193225 김도윤

첨단융합공학부

[github.com/doyun80292-dev/path-compare](https://github.com/doyun80292-dev/path-compare)

2026. 10. 18

# 지도 위에서 가장 싼 길 찾기를 고른 이유

- **실제로 쓰는 문제다.** 내비게이션, 배달 로봇, 자율주행 경로 계획, 게임 캐릭터 이동이 모두 "지도에서 목적지까지 비용이 가장 적은 길"을 찾는다.
- **방식이 여럿이고 보장 조건이 다르다.** BFS · Dijkstra · A\*는 모두 "최단 경로"라고 불리지만, 최적을 보장하는 조건과 뒤흔는 칸 수가 다르다. 이 차이를 직접 재 보고 싶었다.
- **과제 1의 틀을 그대로 쓸 수 있다.** 공통 인터페이스(함수 포인터 표), 연산 횟수 세기, minstd 입력, 테스트 구조를 가져왔고, 과제 1에서 공부한 **힙**을 Dijkstra · A\*의 우선순위 큐로 다시 썼다.
- **결과를 눈으로 볼 수 있다.** 어느 칸을 뒤흔는지 그림으로 그리면 숫자가 왜 그렇게 나왔는지 설명하기 쉽다.

## 과제 1 → 프로젝트

|       | 과제 1          | 프로젝트        |
|-------|---------------|-------------|
| 대상    | 정렬 3종         | 경로 탐색 3종    |
| 세는 것  | 비교 · 이동       | 확장 칸 · 힙 삽입 |
| 입력 모양 | 무작위 · 정렬됨 ... | 지도 5종       |
| 품질    | 안정성           | 경로 비용       |
| 재현    | minstd 씨앗 고정  |             |

과제 1과 같은 주제는 안 되지만 확장은 된다는 안내에 맞춰, 틀만 가져오고 문제는 바꿨다.

# 한 문장으로

칸마다 지나가는 비용이 정해진  $n \times n$  격자 지도에서, 왼쪽 위 칸에서 오른쪽 아래 칸까지 **비용 합이 가장 작은 경로**를 찾는다.

|         |                                                 |
|---------|-------------------------------------------------|
| 이동      | 상하좌우 4방향, 벽(#)은 못 지나감                           |
| 경로 비용   | 들어가는 칸의 비용(1~9) 합. 출발 칸은 안 셈                    |
| 그래프로 보면 | 정점 $V = n^2$ , 간선 $E \approx 4n^2$ , 간선 가중치 1~9 |
| 정답      | 최소 비용. 못 가면 "없음"                                |

|   |   |   |
|---|---|---|
| S | 9 | G |
| 1 | 9 | 1 |
| 1 | 1 | 1 |

2걸음, 비용  $9 + 1 = 10$

|   |   |   |
|---|---|---|
| S | 9 | G |
| 1 | 9 | 1 |
| 1 | 1 | 1 |

6걸음, 비용 6 (최소)

오른쪽 예: 걸음 수가 가장 적은 길과 비용이 가장 적은 길이 다르다. 이  $3 \times 3$  지도는 테스트에도 그대로 들어 있다.

## BFS 너비 우선 탐색 — 걸음 수 순서

- 큐에서 칸을 꺼내고, 아직 안 본 이웃을 뒤에 넣는다. 출발에서 걸음 수가 적은 칸부터 차례로 퍼진다.
- 칸 비용을 **보지 않는다**. 그래서 모든 칸 비용이 같을 때만 최소 비용이고, 비용이 다르면 "걸음 수가 가장 적은 길"을 준다.

|        |                        |
|--------|------------------------|
| 시간     | $O(V + E)$             |
| 추가 메모리 | 칸마다 int 2개 (parent, 큐) |
| 최적 보장  | 비용이 모두 같을 때만           |
| 자료구조   | 큐 (배열 하나로 구현)          |

```

queue ← [S]; parent[S] ← S
while queue not empty:
    v ← queue.pop_front()
    if v = G: return path(parent)
    for u in neighbors(v):      # 오른쪽·아래·왼쪽·위
        if parent[u] = none:
            parent[u] ← v
            queue.push_back(u)
return 없음

```

## Dijkstra 지금까지 비용이 가장 작은 칸부터

- 힙에서 **g(출발부터의 비용)**가 가장 작은 칸을 꺼내 확정하고, 이웃의 g를 줄일 수 있으면 줄여서 힙에 넣는다.
- 비용이 음수만 아니면 꺼내는 순간 그 칸의 g가 최소다 → **항상 최적.**
- 힙에서 값을 고치는 대신 **새로 한 번 더 넣고**, 꺼낼 때 이미 확정된 칸이면 버린다(lazy deletion).

|        |                            |
|--------|----------------------------|
| 시간     | $O((V + E) \log V)$        |
| 추가 메모리 | 칸마다 13바이트 + 힙              |
| 최적 보장  | 비용 $\geq 0$ 이면 항상          |
| 자료구조   | 이진 최소 힙 (과제 1 힙 정렬 코드 재사용) |

```

dist[*] ← ∞; dist[S] ← 0; heap ← {(0, S)}
while heap not empty:
    (f, v) ← heap.pop_min()
    if closed[v]: continue          # 옛 기록
    closed[v] ← true
    if v = G: return path(parent)
    for u in neighbors(v):
        nd ← dist[v] + cost[u]
        if nd < dist[u]:
            dist[u] ← nd; parent[u] ← v
            heap.push((nd, u))      # f = g
return 없음

```

## A\* Dijkstra + 남은 거리 추정 h (새로 공부)

- 우선순위를  $f = g + h$ 로 바꾼다. h는 "목표까지 적어도 이만큼은 남았다"는 추정이라 목표 쪽 칸을 먼저 꺼낸다.
- $h$  = 맨해튼 거리  $\times$  지도의 최소 칸 비용. 4방향이면 그보다 적게 못 움직이고, 한 걸음에 최소 비용은 들므로 **과대평가하지 않는다**(admissible, consistent)  $\rightarrow$  최적 유지.
- $f$ 가 같으면  $g$ 가 큰 칸(목표에 가까운 칸)을 먼저 꺼낸다.

|        |                                        |
|--------|----------------------------------------|
| 시간     | 최악 $O((V + E) \log V)$ , 실제로는 $h$ 에 달림 |
| 추가 메모리 | Dijkstra와 같은 배열 + 힙                    |
| 최적 보장  | $h$ 가 과대평가하지 않으면                       |
| 자료구조   | Dijkstra와 같은 힙 + 휴리스틱                  |

$h(v) = (|x - g_x| + |y - g_y|) \times \text{minCost}$

# Dijkstra와 다른 곳은 이 한 줄뿐:

`heap.push((nd + h(u), nd, u))`    #  $f = g + h$

# 코드에서도 몸통을 하나로 합침

`dijkstraSolve = bestFirstSearch(hUnit = 0)`

`astarSolve = bestFirstSearch(hUnit = minCost)`

기대:  $h$ 가 실제 남은 비용에 가까울수록

덜 뒤진다.  $h = 0$ 이면 Dijkstra와 같다.

# 무엇을, 어떻게 재나

| 기준    | 지표         | 재는 방법                  |
|-------|------------|------------------------|
| 속도    | 실행 시간 (ms) | clock(), 5회 평균         |
|       | 확장한 칸 수    | 꺼내서 이웃을 본 칸. 기계와 무관    |
|       | 힙 · 큐 삽입 수 | lazy deletion 중복까지 셈   |
| 메모리   | 추가 바이트     | 탐색이 malloc한 양 (지도 제외)  |
|       | open 최대 크기 | 큐 · 힙이 가장 컸을 때 원소 수    |
| 결과 품질 | 비용 비       | 찾은 비용 ÷ 최소 비용 (1이면 최적) |
| 입력 변화 | 위 전부       | 지도 5종 × 크기 6단계         |

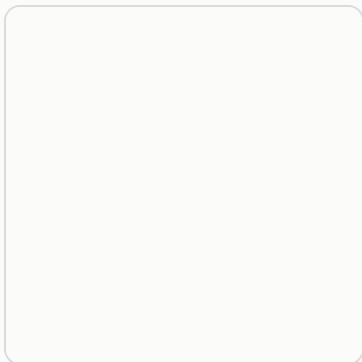
## 같은 조건으로 맞춘 것

- 같은 지도, 같은 출발 · 도착 칸
- 같은 이웃 순서 (오른쪽 · 아래 · 왼쪽 · 위)
- 같은 종료 조건: 도착 칸을 **꺼낼 때**
- 같은 언어 · 옵션: C17, gcc -O2

## 맞는지 확인

- 모든 경로를 pathValid로 검사: 이어짐 · 벽 없음 · 비용 합
- 최소 비용은 Dijkstra와 **따로 짠** 정답 계산기와 대조 (테스트 40개 통과)

# 입력 데이터 · 크기 · 반복



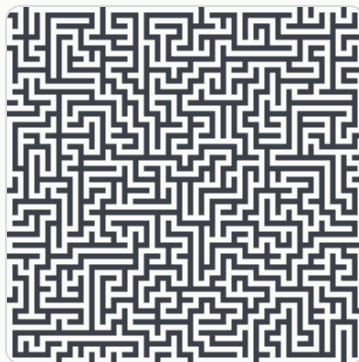
빈 지도

벽 없음, 비용 1



무작위 벽

칸마다 30% 벽



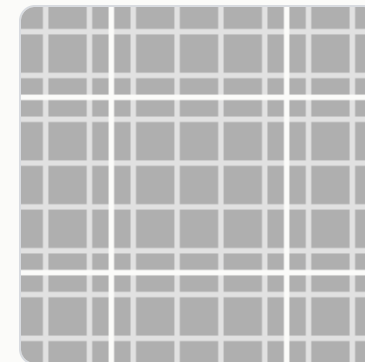
미로

길이 하나뿐인 미로



지형 비용

비용 1~9 무작위



도로망

간선 1 · 골목 3 · 비포장 7

## 지도 5종

## 크기

## 반복

## 재현

비용이 같은 지도 3종(BFS도 최적) + 비용이 다른 지도 2종(BFS가 최적이지 않음). 위 그림은  $65 \times 65$ , 진할수록 비싼 칸

한 번  $n = 65, 129, 257, 513, 1,025, 2,049$  (두 배씩, 미로 때문에 홀수)  $\rightarrow$  4천 ~ 420만 칸

지도별 비교는  $n = 513$ 에서 5회 평균, 크기 실험은 3회 평균. 횡수 지표는 매번 같다

minstd 씨앗 20261018 고정  $\rightarrow$  어느 기계에서나 같은 지도 · 같은 횡수. 시간만 기계마다 다름

## 09 예비 결과 (자유)

# n = 513 (26만 칸)에서 먼저 돌려 본 결과

| 지도    | Dijkstra 확장 | A* 확장  | BFS ms | Dijkstra ms | A* ms | BFS 비용 비 |
|-------|-------------|--------|--------|-------------|-------|----------|
| 빈 지도  | 100.0%      | 0.39%  | 9.88   | 47.7        | 1.25  | 1.00     |
| 무작위 벽 | 69.0%       | 2.0%   | 13.1   | 33.5        | 2.28  | 1.00     |
| 미로    | 20.5%       | 20.4%  | 3.05   | 8.41        | 7.78  | 1.00     |
| 지형 비용 | 100.0%      | 100.0% | 12.9   | 48.6        | 58.5  | 1.77     |
| 도로망   | 100.0%      | 97.9%  | 9.96   | 57.2        | 100.0 | 5.72     |

### A\*가 잘 되는 곳

빈 지도 · 무작위 벽: 확장 칸이 Dijkstra의 2.8% 이하, 시간도 14배 이상 빠름.

### A\*가 오히려 느린 곳

지형 · 도로망: 거의 전부 확장하는데 힙은 더 커져서 Dijkstra보다 느림. 미로에서는 h가 도움이 안 됨.

### BFS의 값

힙이 없어 Dijkstra보다 2.6~5.7배 빠르지만, 비용이 다른 지도에서 경로 비용이 최소의 1.77배 · 5.72배.

초록 = Dijkstra보다 절반 이하, 빨강 = Dijkstra보다 느림 / 최적 아님. 시간은 이 컴퓨터 기준. 확장 % = 확장 칸 ÷ 전체 칸.

# 예비 결과에서 생긴 질문과 최종 제출까지 계획

## 확인할 질문

- 도로망에서 A\*는 왜 느린가?  $n = 2,049$ 에서 A\* 4.1초 vs Dijkstra 1.3초. 확장 칸은 비슷한데 open 최대가 697,387 vs 12,425 (56배) → 힙 크기 탓인지 따로 쟀다.
- $h$ 가 약하면 소용없다. 지형 지도 최소 비용은 2,871인데  $h$ (출발)은 1,024.  $h$ 를 1.5배 · 2배로 키우면 빨라지는 대신 최적이 깨질 수 있다 → 속도-품질 교환을 쟀다.
- 지도 하나로는 결론 못 낸다. 무작위 벽에서 A\* 확장 비율이  $n$ 마다 들쭉날쭉 → 씨앗 10개 평균 · 편차.
- 이론과 측정: 칸 수를 두 배로 할 때 시간 배율을  $O(V \log V)$ 와 견주고, 칸 하나 확장 시간이  $n$ 에 따라 느는 게 캐시 때문인지 본다.

## 지금 상태

- 세 방식 C 구현, 테스트 40개 통과, ASan 통과
- `make run` · `make charts`로 표 · CSV · 그림 재생성
- 저장소: 표지의 GitHub 주소

## AI 사용 내역

A\*의 원리와 admissible · consistent 개념 학습, 코드 · 테스트 작성, 실험 실행, 슬라이드 작성에 Claude(Anthropic)를 사용함. 결과는 테스트와 재실행으로 확인. 질문 · 답변 내역은 최종 보고서에 첨부할 예정.

참고: Hart · Nilsson · Raphael (1968), A\* 원 논문 · Wikipedia "A\* search algorithm", "Dijkstra's algorithm"